

省博物馆参观预约系统后端接口设计

1. 文档说明

1.1 文档目的

本文档用于明确 省博物馆参观预约系统 的 Spring Boot 后端 REST API 设计，作为后续后端核心功能实现、前端 Axios 调用、接口联调、功能测试和性能测试的依据。

本文档重点回答以下问题：

- 前端调用哪些接口
- 接口路径和请求方法是什么
- 请求参数如何命名
- 响应数据结构是什么
- 接口与数据库表如何对应
- 核心预约接口如何保证不超额、不重复
- 后端后续实现优先级是什么

1.2 设计依据

本文档根据以下资料整理：

- 项目要求分析.md
- 项目设计开发流程.md
- 需求拆分.md
- 系统设计文档初稿.md
- 数据库设计.md
- 已完成的正式 Spring Boot 后端项目
- 已完成的 MySQL 数据库 museum_reservation
- 已验证通过的基础接口

1.3 当前后端状态

当前已完成内容：

- 正式 Spring Boot 后端项目已创建
- MySQL 数据源已配置
- schema.sql 已生成并导入本机 MySQL
- 数据库 museum_reservation 已创建
- 统一响应结构 ApiResponse 已实现
- 全局异常处理已实现
- 跨域配置已实现
- 基础接口已运行验证通过

当前已实现接口：

接口	方法	状态	说明
/api/health	GET	已实现	后端健康检查和数据库连通检查
/api/museum/info	GET	已实现	游客端本馆场馆信息

/api/museum/info	GET	已实现	游客端查询场馆信息
/api/notices	GET	已实现	游客端查询启用公告

1.4 设计定位

本文档是后端接口的正式设计稿。

后续开发应优先按照本文档实现接口，不再沿用之前环境验证 Demo 的简化接口和简化表结构。

2. 接口设计总原则

2.1 前后端分离原则

前端不直接访问数据库。

前端通过 Axios 调用 Spring Boot REST API，后端负责：

- 参数校验
- 业务规则判断
- 数据库读写
- 事务控制
- 并发控制
- 统一返回结果

2.2 基础路径约定

所有接口统一以 /api 开头。

类型	路径前缀	说明
通用接口	/api	健康检查、场馆信息、公告等
游客端接口	/api	游客登录、查询时段、提交预约、我的预约
管理员端接口	/api/admin	管理员登录和后台管理

2.3 HTTP 方法约定

方法	用途
GET	查询数据
POST	新增数据或提交业务操作
PUT	修改数据
DELETE	删除或禁用数据

2.4 字段命名约定

后端接口 JSON 字段统一使用 camelCase。

数据库字段使用 snake_case。

示例：

数据库字段	接口字段
-------	------

id_card	idCard
visit_date	visitDate
slot_name	slotName
booked_count	bookedCount
total_capacity	totalCapacity
booking_start	bookingStart
booking_end	bookingEnd
created_at	createdAt
updated_at	updatedAt

2.5 时间格式约定

日期使用：

```
yyyy-MM-dd
```

时间使用：

```
HH:mm:ss
```

日期时间使用后端当前 LocalDateTime 默认序列化格式：

```
yyyy-MM-dd'T'HH:mm:ss
```

后端内部使用 Java 21 时间类型：

- LocalDate
- LocalTime
- LocalDateTime

3. 统一响应格式

3.1 成功响应

所有 JSON 接口统一返回：

```
{
  "success": true,
  "message": "操作成功",
  "data": {}
}
```

字段说明：

字段	类型	说明
success	boolean	是否成功

message	string	提示信息
data	object / array / null	返回数据

当前后端已实现对应类：

```
com.museum.reservation.dto.ApiResponse
```

3.2 失败响应

业务失败返回：

```
{
  "success": false,
  "message": "预约名额已满",
  "data": null
}
```

参数错误返回：

```
{
  "success": false,
  "message": "请求参数不正确",
  "data": null
}
```

系统异常返回：

```
{
  "success": false,
  "message": "系统内部错误",
  "data": null
}
```

当前后端已实现对应类：

```
com.museum.reservation.exception.GlobalExceptionHandler
```

3.3 HTTP 状态码约定

状态码	场景
200	请求成功
400	请求参数错误或业务规则不通过
500	后端未知异常

课程大作业阶段暂不强制细分更多 HTTP 状态码，重点保证接口返回结构统一。

4. 通用接口

4.1 健康检查接口

接口说明

用于检查 Spring Boot 后端是否启动、数据库是否可连接。

请求信息

```
GET /api/health
```

请求参数

无。

响应示例

```
{
  "success": true,
  "message": "操作成功",
  "data": {
    "status": "UP",
    "database": "UP",
    "time": "2026-05-07T11:16:06"
  }
}
```

数据库对应

数据库表	说明
无固定业务表	执行 SELECT 1 验证数据库连通

实现状态

已实现。

5. 游客端接口设计

5.1 游客实名登录

接口说明

游客输入姓名、身份证号和手机号后登录系统。

如果身份证号已存在，则更新游客姓名和手机号；如果不存在，则创建游客记录。

该接口不做复杂账号密码体系，符合课程大作业中的实名预约场景。

请求信息

```
POST /api/visitor/login
```

请求体

```
{
  "name": "张三",
  "idCard": "110101199001010011",
  "phone": "13800138000"
}
```

参数说明

字段	类型	必填	说明
name	string	是	游客姓名
idCard	string	是	身份证号
phone	string	是	手机号

校验规则

- 姓名不能为空
- 身份证号不能为空
- 手机号不能为空
- 身份证号长度建议校验为 18 位
- 手机号建议校验为 11 位数字

响应数据

```
{
  "success": true,
  "message": "登录成功",
  "data": {
    "visitorId": 1,
    "name": "张三",
    "idCard": "110101199001010011",
    "phone": "13800138000"
  }
}
```

数据库对应

数据库表	操作
visitor	根据 id_card 查询、新增或更新游客信息

实现优先级

中。

5.2 查询场馆信息

接口说明

游客端首页查询博物馆基本信息、开放时间和参观须知。

请求信息

GET /api/museum/info

请求参数

无。

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": {
    "id": 1,
    "name": "省博物馆",
    "address": "省会城市文化中心博物馆路 1 号",
    "openInfo": "开放时间：周二至周日 09:00-17:00, 16:30 停止入馆。",
    "rules": "游客需实名预约，按预约日期和时段入馆。",
    "status": 1,
    "updatedAt": "2026-05-07T10:56:58"
  }
}
```

数据库对应

数据库表	操作
<code>museum_info</code>	查询 <code>status = 1</code> 的场馆信息

实现状态

已实现并验证通过。

5.3 查询公告通知

接口说明

游客端查询已启用公告，包括普通通知、特殊展览通知、闭馆提醒和预约规则。

请求信息

GET /api/notices

请求参数

无。

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": [
    {
      "id": 1,
      "title": "参观预约须知",
      "content": "本馆实行实名预约参观，请提前选择参观日期和入场时段。",
      "type": "RULE",
      "enabled": 1,
      "createdAt": "2026-05-07T10:56:58"
    }
  ]
}
```

公告类型

值	含义
NORMAL	普通通知
EXHIBITION	特殊展览
CLOSE	闭馆通知
RULE	预约规则

数据库对应

数据库表	操作
notice	查询 enabled = 1 的公告

实现状态

已实现并验证通过。

5.4 查询可预约时段

接口说明

游客端查询当前可预约日期和分时时段，前端用于展示剩余名额。

请求信息

GET /api/slots

Query 参数

参数	类型	必填	说明
visitDate	string	否	指定参观日期，格式 yyyy-MM-dd

--	--	--	--

如果不传 `visitDate`，默认查询未来可预约的所有启用时段。

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": [
    {
      "slotId": 1,
      "activityId": 1,
      "activityName": "省博物馆参观预约",
      "visitDate": "2026-05-08",
      "slotName": "上午场",
      "startTime": "09:00:00",
      "endTime": "11:00:00",
      "totalCapacity": 50,
      "bookedCount": 0,
      "remaining": 50,
      "enabled": 1,
      "activityStatus": "OPEN",
      "bookingStart": "2026-05-07T09:56:58",
      "bookingEnd": "2026-05-08T23:59:59"
    }
  ]
}
```

查询规则

游客端只应展示满足以下条件的数据：

- 预约活动状态为 `OPEN`
- 预约时段 `enabled = 1`
- 参观日期不早于当前日期
- 剩余名额由 `totalCapacity - bookedCount` 计算

数据库对应

数据库表	操作
reservation_activity	查询预约活动日期、预约开放时间、活动状态
reservation_slot	查询分时时段、总名额、已预约人数

实现优先级

高。

5.5 提交预约

接口说明

游客选择预约时段后提交预约。

该接口是系统最核心接口，必须保证：

- 不能超额预约
- 不能重复预约
- 预约记录与名额数据一致
- 并发请求下数据仍然正确

请求信息

POST /api/reservations

请求体

```
{
  "name": "张三",
  "idCard": "110101199001010011",
  "phone": "13800138000",
  "slotId": 1
}
```

参数说明

字段	类型	必填	说明
name	string	是	游客姓名
idCard	string	是	身份证号
phone	string	是	手机号
slotId	number	是	预约时段 ID

核心校验规则

后端必须按顺序执行以下校验：

1. 校验姓名、身份证号、手机号和 slotId。
2. 查询 reservation_slot，确认时段存在。
3. 查询 reservation_activity，确认活动存在。
4. 校验 reservation_activity.status = OPEN。
5. 校验 reservation_slot.enabled = 1。
6. 校验当前时间不早于 booking_start。
7. 校验当前时间不晚于 booking_end。
8. 校验同一身份证同一参观日期未预约。
9. 校验剩余名额大于 0。
10. 原子更新 reservation_slot.booked_count。
11. 插入 reservation_record。
12. 返回预约编号和二维码内容。

并发控制要求

扣减名额必须使用数据库条件更新：

```
UPDATE reservation_slot
SET booked_count = booked_count + 1,
    version = version + 1
WHERE id = ?
    AND enabled = 1
    AND booked_count < total_capacity;
```

如果受影响行数为 0，说明名额已满或时段不可用，应返回失败。

事务要求

以下操作必须位于同一个事务中：

- 1. 查询游客或创建游客。
- 2. 校验活动和时段。
- 3. 原子扣减 booked_count。
- 4. 插入 reservation_record。

如果插入预约记录失败，必须回滚名额扣减。

响应数据

```
{
  "success": true,
  "message": "预约成功",
  "data": {
    "reservationId": 1,
    "reservationNo": "MR202605080001",
    "visitorName": "张三",
    "idCard": "110101199001010011",
    "phone": "13800138000",
    "visitDate": "2026-05-08",
    "slotName": "上午场",
    "startTime": "09:00:00",
    "endTime": "11:00:00",
    "qrContent": "MR202605080001|110101199001010011|2026-05-08|上午场",
    "createdAt": "2026-05-07T11:30:00"
  }
}
```

常见失败信息

场景	message
参数不完整	请求参数不正确
身份证号格式错误	身份证号格式不正确
手机号格式错误	手机号格式不正确
时段不存在	预约时段不存在

活动未开放	当前预约活动未开放
未到预约开始时间	预约尚未开始
已超过预约结束时间	预约已经结束
重复预约	同一身份证同一天只能预约一次
名额已满	预约名额已满

数据库对应

数据库表	操作
visitor	根据身份证查询或创建游客
reservation_activity	查询活动状态和预约时间规则
reservation_slot	原子扣减 booked_count
reservation_record	插入预约成功记录

关键数据库约束

约束	作用
uk_visitor_id_card	防止游客身份证重复
uk_reservation_record_id_card_visit_date	防止同一身份证同一天重复预约
chk_reservation_slot_booked_count	保证已预约人数不超过总名额

实现优先级

最高。

5.6 查询我的预约记录

接口说明

游客根据身份证号查询自己的预约记录。

请求信息

```
GET /api/reservations/my
```

Query 参数

参数	类型	必填	说明
idCard	string	是	身份证号

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": [
    {
      "reservationId": 1,
      "reservationNo": "MR202605080001",
      "visitorName": "张三",
      "idCard": "110101199001010011",
      "phone": "13800138000",
      "visitDate": "2026-05-08",
      "slotName": "上午场",
      "status": "SUCCESS",
      "qrContent": "MR202605080001|110101199001010011|2026-05-08|上午场",
      "createdAt": "2026-05-07T11:30:00"
    }
  ]
}
```

数据库对应

数据库表	操作
visitor	根据身份证号查询游客
reservation_record	查询该身份证对应的预约记录

实现优先级

中。

6. 管理员端接口设计

6.1 管理员登录

接口说明

管理员输入用户名和密码登录后台。

课程大作业阶段可以暂不引入 Spring Security，先实现简单登录校验。

请求信息

```
POST /api/admin/login
```

请求体

```
{
  "username": "admin",
  "password": "admin123"
}
```

参数说明

字段	类型	必填	说明
username	string	是	管理员用户名
password	string	是	管理员密码

校验规则

- 用户名不能为空
- 密码不能为空
- 管理员必须存在
- 管理员状态必须为启用
- 密码必须匹配

响应数据

```
{
  "success": true,
  "message": "登录成功",
  "data": {
    "adminId": 1,
    "username": "admin",
    "role": "ADMIN"
  }
}
```

数据库对应

数据库表	操作
admin_user	根据 username 查询管理员并校验密码和状态

实现优先级

最高。

6.2 查询后台场馆信息

接口说明

管理员查询场馆信息，用于后台编辑页面回显。

请求信息

```
GET /api/admin/museum/info
```

请求参数

无。

响应数据

与 `/api/museum/info` 基本一致，但后台可以查询到停用状态信息。

数据库对应

数据库表	操作
<code>museum_info</code>	查询场馆信息

实现优先级

高。

6.3 修改场馆信息

接口说明

管理员修改博物馆名称、地址、开放时间、参观须知和状态。

请求信息

```
PUT /api/admin/museum/info
```

请求体

```
{
  "name": "省博物馆",
  "address": "省会城市文化中心博物馆路 1 号",
  "openInfo": "开放时间：周二至周日 09:00-17:00，16:30 停止入馆。",
  "rules": "游客需实名预约，按预约日期和时段入馆。",
  "status": 1
}
```

参数说明

字段	类型	必填	说明
<code>name</code>	string	是	场馆名称
<code>address</code>	string	是	场馆地址
<code>openInfo</code>	string	是	开放时间说明
<code>rules</code>	string	是	参观须知和预约规则
<code>status</code>	number	是	状态，1 正常，0 停用

响应数据

```
{
  "success": true,
  "message": "修改成功",
}
```

```
"data": null
}
```

数据库对应

数据库表	操作
<code>museum_info</code>	更新场馆信息

实现优先级

高。

6.4 查询公告列表

接口说明

管理员查询所有公告，包括启用和禁用公告。

请求信息

```
GET /api/admin/notices
```

Query 参数

参数	类型	必填	说明
<code>enabled</code>	number	否	1 启用，0 禁用，不传查询全部
<code>type</code>	string	否	公告类型

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": [
    {
      "id": 1,
      "title": "参观预约须知",
      "content": "本馆实行实名预约参观。",
      "type": "RULE",
      "enabled": 1,
      "createdAt": "2026-05-07T10:56:58",
      "updatedAt": "2026-05-07T10:56:58"
    }
  ]
}
```

数据库对应

--	--

数据库表	操作
notice	查询公告列表

实现优先级

高。

6.5 新增公告

请求信息

POST /api/admin/notices

请求体

```
{
  "title": "临时闭馆通知",
  "content": "因设备维护，本馆某日临时闭馆。",
  "type": "CLOSE",
  "enabled": 1
}
```

参数说明

字段	类型	必填	说明
title	string	是	公告标题
content	string	是	公告内容
type	string	是	NORMAL、EXHIBITION、CLOSE、RULE
enabled	number	是	1 启用，0 禁用

响应数据

```
{
  "success": true,
  "message": "新增成功",
  "data": {
    "id": 4
  }
}
```

数据库对应

数据库表	操作
notice	新增公告

实现优先级

高。

6.6 修改公告

请求信息

```
PUT /api/admin/notices/{id}
```

Path 参数

参数	类型	必填	说明
id	number	是	公告 ID

请求体

```
{
  "title": "入馆提醒",
  "content": "预约成功后请按预约时段入馆。",
  "type": "NORMAL",
  "enabled": 1
}
```

响应数据

```
{
  "success": true,
  "message": "修改成功",
  "data": null
}
```

数据库对应

数据库表	操作
notice	根据 ID 修改公告

实现优先级

高。

6.7 删除公告

接口说明

管理员删除公告。

课程大作业阶段可以采用物理删除，也可以将 enabled 改为 0。为降低复杂度，建议优先采用逻辑禁用。

请求信息

```
DELETE /api/admin/notices/{id}
```

Path 参数

参数	类型	必填	说明
id	number	是	公告 ID

响应数据

```
{
  "success": true,
  "message": "删除成功",
  "data": null
}
```

数据库对应

数据库表	操作
notice	建议更新 enabled = 0

实现优先级

中。

6.8 查询预约活动

接口说明

管理员查询预约活动列表，包括参观日期、每日名额、预约开放时间和活动状态。

请求信息

```
GET /api/admin/activities
```

Query 参数

参数	类型	必填	说明
status	string	否	活动状态
startDate	string	否	开始参观日期
endDate	string	否	结束参观日期

响应数据

```
{
```

```
"success": true,
"message": "操作成功",
"data": [
  {
    "id": 1,
    "activityName": "省博物馆参观预约",
    "visitDate": "2026-05-08",
    "dailyCapacity": 200,
    "personLimit": 1,
    "bookingStart": "2026-05-07T09:56:58",
    "bookingEnd": "2026-05-08T23:59:59",
    "status": "OPEN",
    "createdAt": "2026-05-07T10:56:58",
    "updatedAt": "2026-05-07T10:56:58"
  }
]
```

活动状态

值	含义
DRAFT	草稿，暂不开放
OPEN	启用，游客端可按时间规则预约
CLOSED	已关闭，不允许预约
DISABLED	禁用，不允许预约

数据库对应

数据库表	操作
reservation_activity	查询预约活动列表

实现优先级

高。

6.9 新增预约活动

请求信息

POST /api/admin/activities

请求体

```
{
  "activityName": "省博物馆参观预约",
  "visitDate": "2026-05-09",
  "dailyCapacity": 200,
```

```
"personLimit": 1,
"bookingStart": "2026-05-08 08:00:00",
"bookingEnd": "2026-05-09 23:59:59",
"status": "OPEN"
}
```

校验规则

- 参观日期不能为空
- 同一参观日期只能有一个预约活动
- 每日总名额必须大于 0
- 单人预约上限必须大于 0
- 预约结束时间必须晚于预约开始时间
- 状态必须属于 DRAFT、OPEN、CLOSED、DISABLED

响应数据

```
{
  "success": true,
  "message": "新增成功",
  "data": {
    "id": 4
  }
}
```

数据库对应

数据库表	操作
reservation_activity	新增预约活动

实现优先级

高。

6.10 修改预约活动

请求信息

```
PUT /api/admin/activities/{id}
```

Path 参数

参数	类型	必填	说明
id	number	是	预约活动 ID

请求体

```
{
```

```
"activityName": "省博物馆参观预约",
"visitDate": "2026-05-09",
"dailyCapacity": 200,
"personLimit": 1,
"bookingStart": "2026-05-08 08:00:00",
"bookingEnd": "2026-05-09 23:59:59",
"status": "OPEN"
}
```

重要规则

如果该活动下已经存在预约记录，修改参观日期和大幅调整名额需要谨慎。

课程大作业阶段建议规则：

- 可以修改活动名称、预约开始时间、预约结束时间、状态
- 修改每日总名额时，不应小于该活动下所有时段已预约人数之和
- 如果已经产生预约记录，不建议修改参观日期

响应数据

```
{
  "success": true,
  "message": "修改成功",
  "data": null
}
```

数据库对应

数据库表	操作
reservation_activity	根据 ID 修改预约活动

实现优先级

高。

6.11 查询预约时段

接口说明

管理员查询预约时段和名额情况。

请求信息

GET /api/admin/slots

Query 参数

参数	类型	必填	说明
activityId	number	否	预约活动 ID

visitDate	string	否	参观日期
-----------	--------	---	------

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": [
    {
      "id": 1,
      "activityId": 1,
      "visitDate": "2026-05-08",
      "slotName": "上午场",
      "startTime": "09:00:00",
      "endTime": "11:00:00",
      "totalCapacity": 50,
      "bookedCount": 0,
      "remaining": 50,
      "enabled": 1,
      "version": 0
    }
  ]
}
```

数据库对应

数据库表	操作
reservation_slot	查询预约时段
reservation_activity	可关联查询活动信息

实现优先级

高。

6.12 新增预约时段

请求信息

```
POST /api/admin/slots
```

请求体

```
{
  "activityId": 1,
  "slotName": "上午场",
  "startTime": "09:00:00",
  "endTime": "11:00:00",
  "totalCapacity": 50,
```

```
"enabled": 1
}
```

校验规则

- 预约活动必须存在
- 结束时间必须晚于开始时间
- 时段总名额必须大于 0
- 同一活动下不能重复配置相同开始和结束时间的时段
- 同一活动下所有启用时段名额之和不应超过活动每日总名额

响应数据

```
{
  "success": true,
  "message": "新增成功",
  "data": {
    "id": 13
  }
}
```

数据库对应

数据库表	操作
reservation_activity	查询活动日期和每日总名额
reservation_slot	新增预约时段

实现优先级

高。

6.13 修改预约时段

请求信息

```
PUT /api/admin/slots/{id}
```

Path 参数

参数	类型	必填	说明
id	number	是	预约时段 ID

请求体

```
{
  "slotName": "上午场",
  "startTime": "09:00:00",
  "endTime": "11:00:00",
}
```



```
"totalCapacity": 50,
"enabled": 1
}
```

重要规则

- 结束时间必须晚于开始时间
- 新总名额不能小于当前 bookedCount
- 已产生预约记录后，不建议随意修改时段时间
- 禁用时段后，游客端不再展示该时段

响应数据

```
{
  "success": true,
  "message": "修改成功",
  "data": null
}
```

数据库对应

数据库表	操作
reservation_slot	修改预约时段

实现优先级

高。

6.14 查询预约名单

接口说明

管理员查询预约记录，可按日期、时段、身份证号和状态筛选。

请求信息

GET /api/admin/reservations

Query 参数

参数	类型	必填	说明
visitDate	string	否	参观日期
slotId	number	否	预约时段 ID
idCard	string	否	身份证号
status	string	否	预约状态

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": [
    {
      "reservationId": 1,
      "reservationNo": "MR202605080001",
      "visitorName": "张三",
      "idCard": "110101199001010011",
      "phone": "13800138000",
      "visitDate": "2026-05-08",
      "slotName": "上午场",
      "status": "SUCCESS",
      "createdAt": "2026-05-07T11:30:00"
    }
  ]
}
```

数据库对应

数据库表	操作
reservation_record	查询预约记录
visitor	关联查询游客姓名
reservation_slot	可关联查询时段详情

实现优先级

中。

6.15 导出预约名单

接口说明

管理员导出预约名单。

课程大作业阶段可以先返回 CSV 文本或 Excel 文件。为降低实现复杂度，建议第一版返回 CSV。

请求信息

GET /api/admin/reservations/export

Query 参数

参数	类型	必填	说明
visitDate	string	否	参观日期
slotId	number	否	预约时段 ID

响应形式

文件下载。

推荐响应头：

```
Content-Type: text/csv;charset=UTF-8
Content-Disposition: attachment; filename=reservations.csv
```

CSV 字段

预约编号,游客姓名,身份证号,手机号,参观日期,预约时段,预约状态,创建时间

数据库对应

数据库表	操作
reservation_record	查询预约记录
visitor	关联查询游客姓名

实现优先级

低。

6.16 后台统计

接口说明

管理员后台首页展示统计数据。

请求信息

```
GET /api/admin/summary
```

Query 参数

参数	类型	必填	说明
visitDate	string	否	参观日期，不传则统计全部或当天

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": {
    "visitorCount": 12,
    "reservationCount": 30,
    "totalCapacity": 600,
    "bookedCount": 30,
    "remaining": 570,
    "slotSummary": [
```

```
{
  "slotId": 1,
  "slotName": "上午场",
  "visitDate": "2026-05-08",
  "totalCapacity": 50,
  "bookedCount": 10,
  "remaining": 40
}
```

数据库对应

数据库表	操作
visitor	统计游客数
reservation_record	统计预约记录数
reservation_slot	统计总名额、已预约人数和剩余名额

实现优先级

中。

6.17 系统状态查看

接口说明

管理员查看系统状态，包括数据库状态、预约请求数量、成功失败数量等。

请求信息

GET /api/admin/status

响应数据

```
{
  "success": true,
  "message": "操作成功",
  "data": {
    "systemStatus": "UP",
    "databaseStatus": "UP",
    "totalRequestCount": 100,
    "successRequestCount": 95,
    "failedRequestCount": 5,
    "lastRequestTime": "2026-05-07T11:30:00"
  }
}
```

数据库对应

数据库表	操作
request_log	可选，统计预约请求情况

实现优先级

低。

7. 接口与数据库表对应关系

功能模块	接口	主要数据库表
健康检查	GET /api/health	无固定业务表
场馆信息展示	GET /api/museum/info	museum_info
公告展示	GET /api/notices	notice
游客实名登录	POST /api/visitor/login	visitor
查询可预约时段	GET /api/slots	reservation_activity, reservation_slot
提交预约	POST /api/reservations	visitor, reservation_activity, reservation_slot, reservation_record
我的预约	GET /api/reservations/my	visitor, reservation_record
管理员登录	POST /api/admin/login	admin_user
场馆信息管理	/api/admin/museum/info	museum_info
公告管理	/api/admin/notices	notice
预约活动管理	/api/admin/activities	reservation_activity
预约时段管理	/api/admin/slots	reservation_slot, reservation_activity
预约名单管理	/api/admin/reservations	reservation_record, visitor
后台统计	/api/admin/summary	visitor, reservation_record, reservation_slot
系统状态	/api/admin/status	request_log

8. 核心业务接口实现要求

8.1 预约接口事务边界

POST /api/reservations 必须开启事务。

推荐 Service 方法：

```
ReservationService.createReservation()
```

事务中必须包含：

1. 查询或创建 visitor。
2. 查询 reservation_slot。
3. 查询 reservation_activity。
4. 校验预约规则。
5. 更新 reservation_slot.booked_count。
6. 插入 reservation_record。

8.2 防止超额预约

不能先查询剩余名额再单独更新。

必须使用条件更新：

```
UPDATE reservation_slot
SET booked_count = booked_count + 1,
    version = version + 1
WHERE id = ?
    AND enabled = 1
    AND booked_count < total_capacity;
```

判断依据：

影响行数 = 1：扣减成功
影响行数 = 0：名额已满或时段不可用

8.3 防止重复预约

数据库已存在唯一约束：

```
reservation_record(id_card, visit_date)
```

后端也应提前查询并提示：

同一身份证同一天只能预约一次

即使并发请求绕过前置查询，数据库唯一约束也能兜底。

8.4 预约编号生成规则

建议格式：

MR + yyyyMMdd + 4位序号

示例：

MR202605080001

课程大作业阶段也可以使用：

MR + 当前时间戳 + 随机数

但必须保证 reservation_no 唯一。

8.5 二维码内容规则

二维码内容可以先保存为普通字符串。

建议格式：

预约编号|身份证号|参观日期|预约时段

示例：

MR202605080001|110101199001010011|2026-05-08|上午场

前端后续可根据该字符串生成二维码。

9. 后端代码实现建议

9.1 包结构建议

当前后端基础包结构已经创建。

后续建议继续使用：

```
com.museum.reservation
├── config
├── controller
├── dto
├── entity
├── exception
├── repository
└── service
```

9.2 Controller 设计建议

建议按模块拆分 Controller：

Controller	负责接口
HealthController	/api/health
MuseumController	游客端场馆和公告查询
VisitorController	游客登录
ReservationController	游客查询时段、提交预约、我的预约
AdminAuthController	管理员登录
AdminMuseumController	后台场馆信息管理
AdminNoticeController	后台公告管理
AdminActivityController	后台预约活动管理

AdminSlotController	后台预约时段管理
AdminReservationController	后台预约名单、导出、统计

9.3 Service 设计建议

业务逻辑必须放在 Service，不应堆在 Controller 中。

Service	负责业务
SystemService	健康检查、系统状态
MuseumService	场馆信息和公告查询
VisitorService	游客登录和信息维护
ReservationService	查询时段、提交预约、我的预约
AdminService	管理员登录
AdminMuseumService	后台场馆信息维护
AdminNoticeService	后台公告维护
AdminActivityService	预约活动维护
AdminSlotService	预约时段维护
StatisticsService	后台统计

9.4 DTO 设计建议

请求 DTO 命名：

```
XxxRequest
```

响应 DTO 命名：

```
XxxResponse
```

示例：

```
AdminLoginRequest
AdminLoginResponse
VisitorLoginRequest
VisitorLoginResponse
ReservationCreateRequest
ReservationCreateResponse
SlotResponse
```

9.5 参数校验建议

后端已引入 Validation 依赖。

后续请求 DTO 建议使用：

```
@NotBlank
```


@NotNull
@Pattern
@Min

常见校验：

字段	校验
name	非空
idCard	非空，18 位
phone	非空，11 位数字
slotId	非空，大于 0
title	非空
totalCapacity	大于 0
bookingEnd	晚于 bookingStart

10. 后续实现优先级

根据项目开发流程和当前后端状态，后续建议按以下顺序实现：

- 1. POST /api/admin/login
- 2. GET /api/admin/museum/info
- 3. PUT /api/admin/museum/info
- 4. GET /api/admin/notices
- 5. POST /api/admin/notices
- 6. PUT /api/admin/notices/{id}
- 7. DELETE /api/admin/notices/{id}
- 8. GET /api/admin/activities
- 9. POST /api/admin/activities
- 10. PUT /api/admin/activities/{id}
- 11. GET /api/admin/slots
- 12. POST /api/admin/slots
- 13. PUT /api/admin/slots/{id}
- 14. POST /api/visitor/login
- 15. GET /api/slots
- 16. POST /api/reservations
- 17. GET /api/reservations/my
- 18. GET /api/admin/reservations
- 19. GET /api/admin/summary
- 20. GET /api/admin/reservations/export
- 21. GET /api/admin/status

其中最高优先级接口是：

POST /api/admin/login
GET /api/slots
POST /api/reservations

因为它们分别对应：

- 后台入口
- 游客查看可预约时段
- 核心预约并发业务

11. 与评分点对应关系

评分点	接口设计支撑
前后端功能完整性	游客端和管理员端接口完整覆盖系统功能
高并发与稳定性	POST /api/reservations 明确原子扣减和事务要求
数据一致性	明确唯一约束、事务边界和数据库条件更新
虚拟机部署	接口均为 REST API，适合部署到 Linux 虚拟机后由浏览器访问
文档规范	本文档可作为后端接口说明和前后端联调依据

12. 当前结论

本文档完成后，后端开发可以从接口设计进入正式业务实现阶段。

当前已经具备以下基础：

正式 Spring Boot 后端项目
MySQL 数据源配置
正式数据库 museum_reservation
统一响应格式
全局异常处理
基础接口验证
后端接口设计文档

下一步建议优先实现：

POST /api/admin/login

然后按本文档顺序逐步完成后台管理接口、游客端预约接口和核心并发预约逻辑。